

Computer and Network Security — Lecture 15

Dr. Karim Lounis

Jan - May 2025



Operating System Security

Overview: What is an Operating System?

Definition

It is a set of programs (called **modules**) that cooperate for the good management and use of **computer resources**. These modules can be classified into two classes, primary (kernel) and secondary.

• Primary modules (Kernel):

- Process manager module.
- Memory manager module.
- File system module.
- Interrupt handling module.
- Other?

• Secondary modules:

- Drivers.
- Networking module.
- Security manager module.
- The Shell (cmd, gui, both)
- Others?

✚ There are various types of operating systems: real-time, mobile, single-taking, multi-tasking, embedded, distributed, network-based, etc.

✚ Some are open-source, whereas other are closed-source operating systems.

✚ Some operating systems provide much better security than others.

Overview: Operating Systems Security

Definition

Operating Systems Security. Is the discipline that studies how to secure operating systems from security threats.

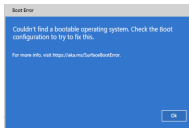
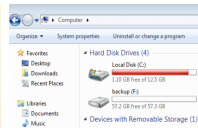
Why do we need a secure operating system, or to secure our operating system?

- Operating systems often store and manage critical data, including user **credential** **financial information**, and **personal files**. **Strong OS security safeguards this data from unauthorized access or modification.**
- Security threats can disrupt **system functionality**, leading to crashes, data loss, or denial-of-service attacks. **Robust OS security helps ensure system stability, availability, and automatic fault-recovery.**
- A well-secured operating system minimizes the risk of malware infections, unauthorized access attempts, and other security incidents.

Overview: Operating Systems Security Aspect

Operating System Aspect. Securing operating systems is crucial too:

- Install regular updates and security patches, & disable unnecessary services.
- Secure user accounts with username, password, and access rights.
- Install anti-malware (e.g., anti-spyware and anti-viruses).
- Implement strong authentication (as per the standard guidelines).
- Audit and log events to track and monitor users' activities.
- Use security software such as firewalls to protect from outsiders.
- Backup and recovery plan — incident response plan.
- Never use cracked operating systems or software. There is no liability.
- Do not plug unverified and untrusted devices (e.g., USB, power cable, etc) that you find abandoned or been given during a social event.



Overview: Operating Systems Security Aspect

Operating System Aspect. Securing operating systems is crucial too:

- Install regular updates and security patches. & disable unnecessary services.

Operating systems, often downloaded from unofficial sources or untrustworthy websites, were later found to be malicious.

Linux Mint Website Hacked; ISO Downloads Replaced with a Backdoor

22 février 2016



The systems of users who downloaded Linux Mint on February 20 may be at risk after it was discovered that Hackers from Sofia, Bulgaria managed to hack into Linux Mint, currently **one of the most popular Linux distributions** available. According to Linux Mint's report, the hacker tricked users into downloading a version of Linux Mint ISO with a backdoor installed by replacing the download links on the site. The link leads to one of their servers offering malicious ISO images of the Linux Mint 17.3 Cinnamon edition. The website has been down since February 21, Sunday, resulting in the loss of thousands of downloads.



Ubuntu Spyware: What to Do?

by [Richard Stallman](#)

Since [Ubuntu version 16.04](#), the spyware search facility is now disabled by default. It appears that the campaign of pressure launched by this article has been partly successful. Nonetheless, offering the spyware search facility as an option is still a problem, as explained below. Ubuntu should make the network search a command users can execute from time to time, not a semipermanent option for users to enable (and probably forget).

Even though the factual situation described in the rest of this page has partly changed, the page is still important. This example should teach our community not to do such things again, but in order for that to happen, we must continue to talk about it.

One of the major advantages of free software is that the community protects users from malicious software. Now Ubuntu [GNU/Linux](#) has become a counterexample. What should we do?

Proprietary software is associated with malicious treatment of the user: surveillance code, digital handcuffs (DRM or Digital Restrictions Management) to restrict users, and back doors that can do nasty things under remote control. Programs that do any of these things are malware and should be treated as such. Widely used

Operating Systems' Fundamental Security Services

Definition

Security Services. Services provided through security mechanisms to secure operating systems and make them resilient against attacks.

- **Authentication.** Prove your identity: What you know/have/are?
- **Confidentiality.** Protect information (stored and processed in the operating system) from being disclosed to unauthorized parties.
- **Integrity.** Protect information (data and operating system's files) from being modified while being stored or processed.
- **Availability.** Guarantee that operating system resources are available at all time. The system can auto-recover following a crash.
- **Non-repudiation.** Nobody can deny having performed an action while using the operating system.

Others: Authorization, Privacy, Trust, ..., Accountability.

Operating Systems Security Threats

The operating system has various security threats:

- **Configuration & Programming Errors.** Include use of weak passwords, default security configs, programming errors, bad updates, ...
 e.g., password cracking, firewall bypass (due to misconfiguration), buffer overflow, integer overflow, deadlocks,
- **Denial of Service.** Flooding, Mail-bombing, Nuke, exploits, ...e.g., Ping of death, ICMP smurf, TCP Syn flood, Zip-Bombing, ...
- **Squatting.** Illegal hosting, bots, ... building botnet (master/slave) for DDoS, storing “illegal” videos on target computer...
- **Malware.** Viruses, worms, trojan, spyware, ransomware, backdoor, ... e.g., Spyeye, Zotob, Iloveyou, WanaCry, Blaster, MyDoom, Stuxnet, ...
- **Spoofing.** IP spoofing, MAC spoofing, Hostname spoofing, ...
- **Social Engineering.** Phishin' (x@y, QR-code...), smishin', & vishin'.
- **Undesirables.** Spams, pop-ups, popunders, hoax, ...
- **Reconnaissance.** Port scanning, OS fingerprinting, footprinting...

Malware

Definition

Malware. Is a program designed to cause harm to an operating system.

Malware differ on how they are propagated and how they infect systems:

- Exploiting a vulnerability in the operating system (0-day-vulnerability).
- Exploiting a vulnerability in a software.
- Being transfered from a storage medium (e.g., USB pendrive).
- Clicking on an attached infected file (e.g., spam).
- Downloading an infected file/software from the Internet.
- Trapped webpage (install this to display the content)
- Being sent by itself from another system.

There r $\neq$ types of malware that threaten operating systems security:

Viruses, Worms, Torjan horse, Bots, Rootkit, Ransomware, Spyware,
Keylogger, Backdoor, macros, mineware (cryptojacking malware),
malicious compiler, [pop-ups, hoax]

Types of Malware

Malicious programs that can cause damage to an operating system:

Virus. Attaches itself to a file or a program enabling it to spread from one computer to another (need human action). It stays wise till it gets triggered.

Worm. Does not infect files, but has the ability to (1) duplicate itself in a system and (2) send a copy of itself through the network (e.g., exploits network traffic to travel). So it can spread without human involvement.

Trojan horse. It comes in the form of a nicely embellished and useful software, but once installed, it actions the malicious function. Generally used to setup backdoors, virus, ransomware, ...

Bot. Runs on user operating systems (Zombie) and waits for commands from a remote attacker which controls it.

Rootkit. Installed after taking over a computer. It is used for erasing evidences to avoid being backtracked.

Types of Malware

Malicious programs that can cause damage to a computer system:

Ransomware. Threatens victims to publish their private data or perpetually block access to it unless a ransom is paid.

Spyware. Used to gather information about users activity over the Internet (profiling) or with respect to a particular software.

Keylogger. A.k.a, Keystroke logger. Used to intercept what users type on their keyboard (can be a hardware as well).

Backdoor. Sets up a listened on the victim system so that the attacker can come back anytime later when needed.

Pop-up. Unsolicited web windows that pops up to display an advertisement or a warning covering the needed web page. Sometimes those windows appear in the back of the main page (pop-under).

Hoax. Jokes, false news, or bad advertisement about a company that is spread among people.

Trap door

Definition

Trap door. Is a piece of code “intentionally” inserted into a program (e.g., an operating system) by the developer for maintainability purposes, bypassing normal security checking procedures.

They would insert a piece of code (trap door) inside a program to be exploited later on and bypass checking procedures.

Example.

```
v=check_validity(logname, password);  
if(v) execute_shell(logname);
```

The insider modifies the code so that it becomes:

```
v=check_validity(logname, password);  
if(v || (strcmp(logname,"RUN")==0)) execute_shell(logname);
```

Periodically check and review the login program files (open-source only).

Operating Systems' Protection Mechanisms and Tools

To protect your operating system, there exist different protection tools:

- ➦ **Anti-virus.** Detect and eliminate different types of malware: signature-based (requires database update), sandbox, and behavior-based.
- ➦ **Firewall.** Software barrier to filter (and block) incoming traffic and also outgoing traffic. Users can define customized filtering rules.
- ➦ **Anti-spyware.** Looks for spyware, trojans, and rootkits.
- ➦ **Integrity Control.** Compare system's integrity with a previous state. It relies on the applying build-in hash functions on system files.
- ➦ **IDS (Intrusion Detection Systems).** Detect intrusion attempts and notify system administrator.
- ➦ **IPS (Intrusion Prevent Systems).** Detect intrusion attempts and notify system administrator, but also actuates a prevention mechanism.
- ➦ **Event Logger.** Log events for further security analysis.

Access Protection in Operating System

Definition

Access Protection. It refers to the set of mechanisms used for controlling the access of processes or users to the computer resources.

Many companies possess valuable information which access needs to be protected, such as technical (a new chip design or software), commercial (studies of the competition or marketing plans), financial (plans for stock offering), or legal (document about potential merger or takeover).

Physical protection — protecting physical access to computers themselves

Home computers increasingly have valuable information on them, too, such as: tax return, credit card numbers, photos, videos, and other private info.

The operating system requires logical protection.

Access Protection in Operating System (Logical)

Security properties in the context of operating systems:

Authentication. Users can prove to the OS who they claim to be:

e.g., password, access card, biometrics.

Data Confidentiality. The owner of a file should be able to specify who can read, copy, and/or execute that file. Some OS provide:

RWX access rights and file encryption.

Data Integrity. The owner of a file should be able to specify who can modify (write, delete, erase content) a particular file. OSs use:

WX access rights and file checksum (hashing)

Access Protection in Operating System (Logical)

Security properties in the context of operating systems:

System Availability. The OS should apply mechanisms to make sure that nobody can disturb the system to make it unusable.

e.g., anti-malware, HIDS, firewall, and access rights, automatic system recovery after system crash (e.g., MINIX).

Authorization. The OS controls access made by authenticated entities (users, processes, ...) to make sure they are only accessing resources on which they have the right to access.

e.g., memory protection, code execution on the stack, privileged code, ...

Users Privacy and Anonymity. The OS protects individuals from misuse of information about them.

e.g., open source OS and software, VPNs (e.g., Tor Browser).

Operating Systems Classic Attacks

Memory Overflow Attacks

Buffer overflow

Definition

Buffer overflow. Is an attack that exploits a bug in a program to cause the bugged program jump to an arbitrary program for execution.

Used by malicious parties to gain unauthorized access to the system or privilege escalation.

Could be run on the stack or on the heap.

Overflow on the stack.

Consists of feeding (inputting) to a program more data than the program expects and forcing it to overwrite the return address on the stack with the address of an exploit code (a.k.a., shellcode) so that the latter will be executed when the return address is popped back to the PC register.

Vulnerability: No C compiler does array bound checking.

Buffer overflow on the stack

Definition

Stack. Is a portion of memory (a.k.a., stack segment) that belongs to a particular process and used mainly for (1) Interrupts and (2) Function calls.

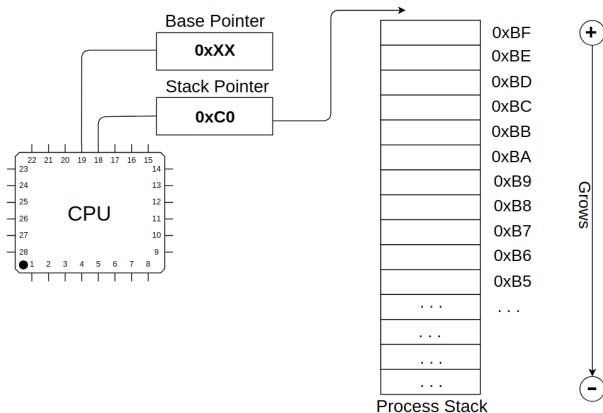
For each process, its dedicated stack is pointed out by the SP (Stack Pointer) and the BP (Base Pointer).

➦ **SP (Stack Pointer).** Is used to move within the stack. It always points to the top of the stack.

➦ **BP (Base Pointer).** Is used to locate the current stack frame as well as for referencing function parameters, local variables, and old base pointers.

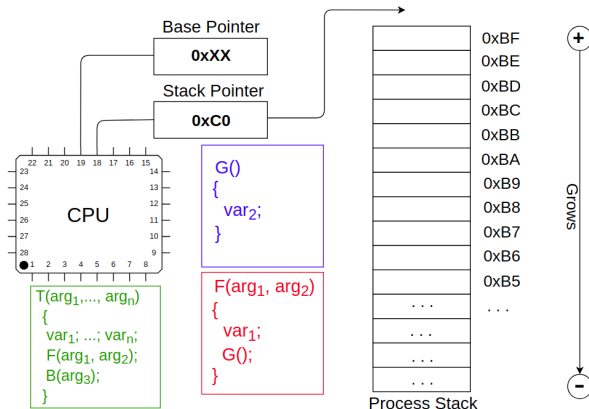
At each function call, a stack frame will be allocated within the stack of the process to host the function local variables, formal parameters, and two important values: **Return address** and the **Old frame pointer**.

Buffer overflow on the stack



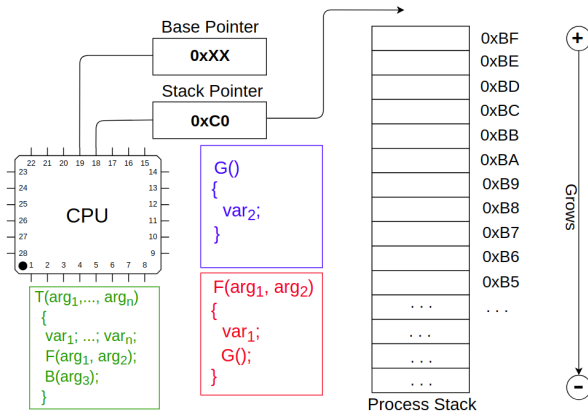
Consider a stack segment that belongs to a given process. Initially, the Base pointer points to a location in the current stack frame whereas the Stack pointer points the top of the stack.

Buffer overflow on the stack



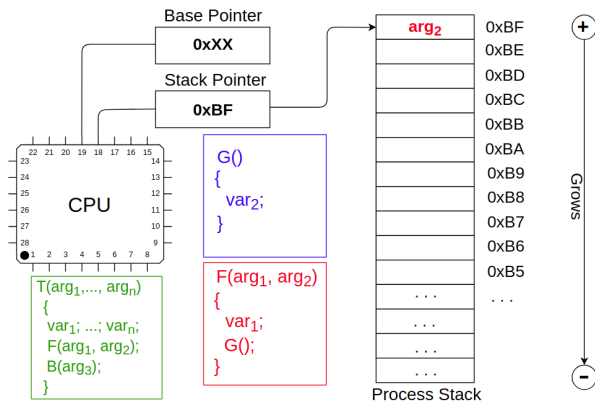
Consider the program code of that process to be, essentially, made of one main function `F()` and one sub-functions `G()`.

Buffer overflow on the stack



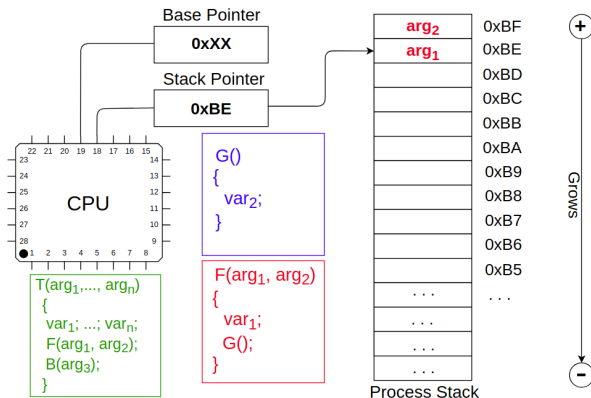
At each function call, a stack function frame is allocated for that function. The size of that frame is computed at program compilation.

Buffer overflow on the stack



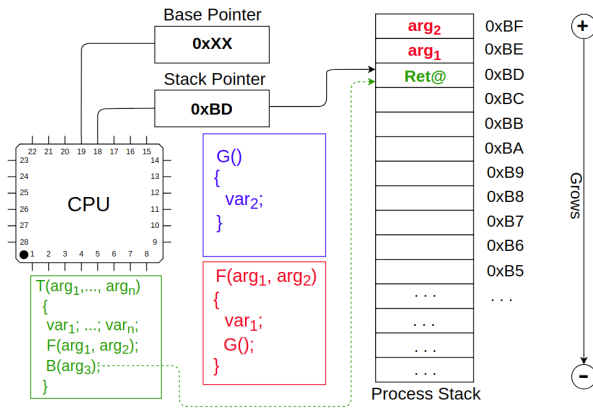
When function F is invoked, the CPU starts pushing the function arguments in reverse order (convention).

Buffer overflow on the stack



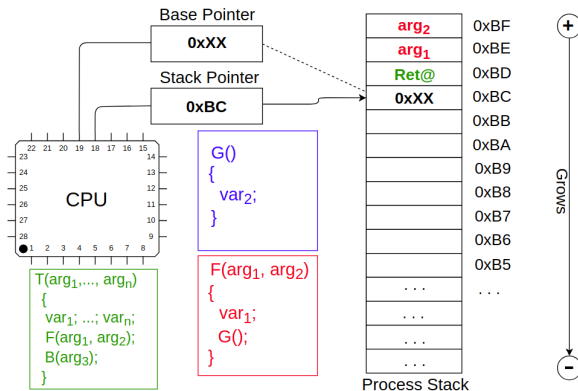
When function **F** is invoked, the CPU starts pushing the function arguments in reverse order (convention).

Buffer overflow on the stack



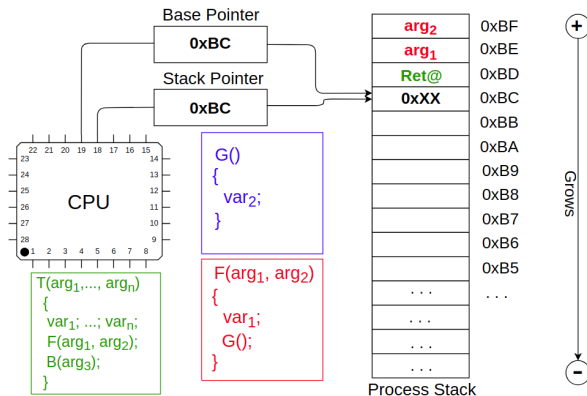
Then, it pushes the return address which is the address of the next instruction following that function call.

Buffer overflow on the stack



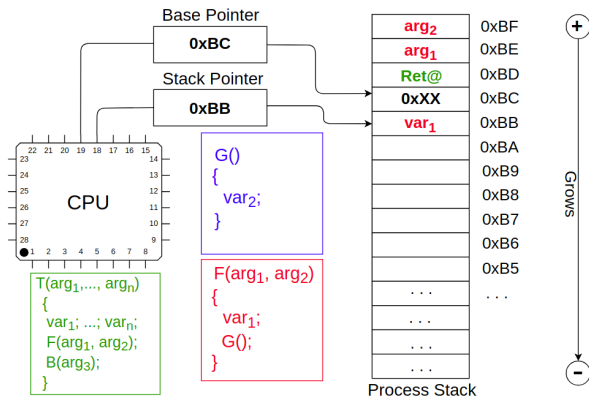
Next, it pushes the BP pointer (to keep track of the previous frame).

Buffer overflow on the stack



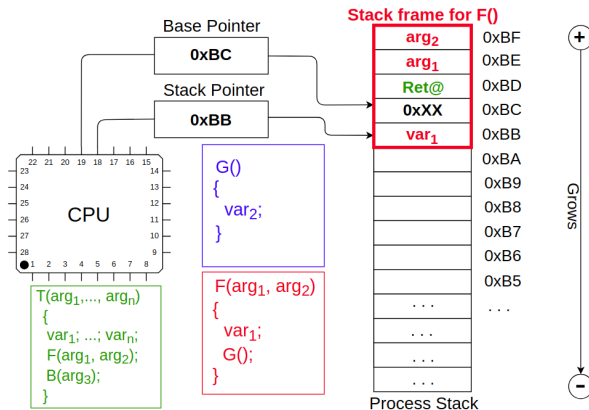
And set a new value for the Base pointer. Now the BP points to a location that allows to perform referencing within the function stack frame.

Buffer overflow on the stack



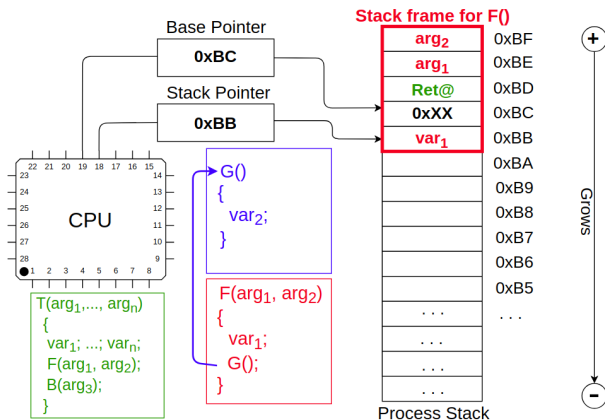
Next, the local variables are pushed in their order of declaration (but last byte first — in reverse). In this case there is only one local variable `var1`.

Buffer overflow on the stack



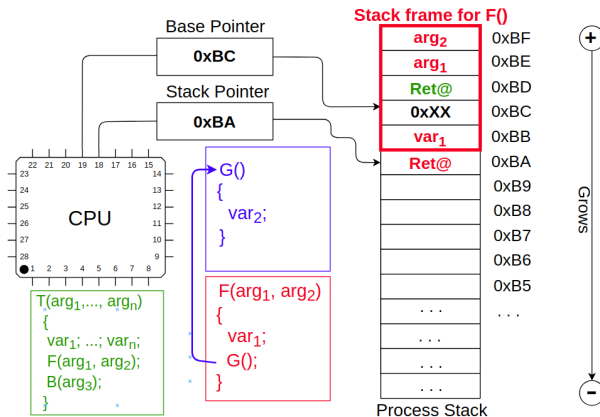
At this stage, the stack frame of function `F()` is complete. To access the arguments or the local variables, the BP pointer is used.

Buffer overflow on the stack



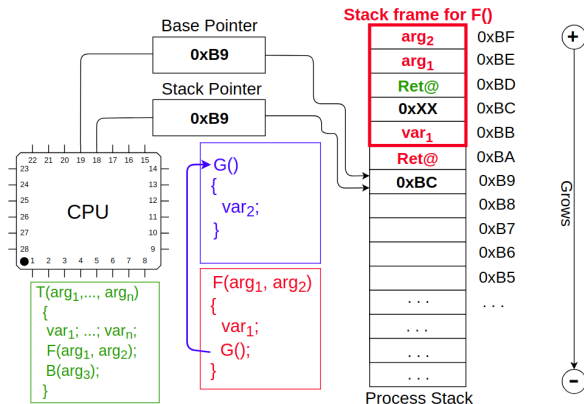
Now, let us consider a function call `G()` inside function `F()`.

Buffer overflow on the stack



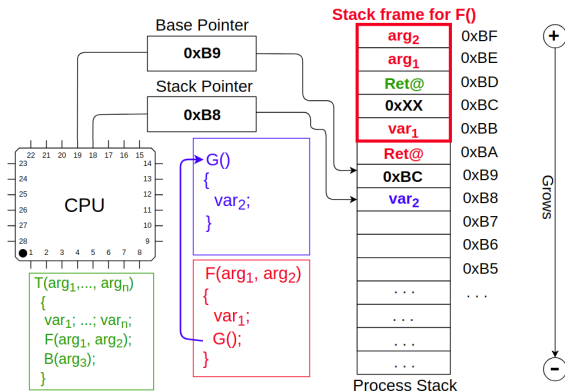
Function `G()` has no arguments. Therefore, the return address is first pushed onto the stack.

Buffer overflow on the stack



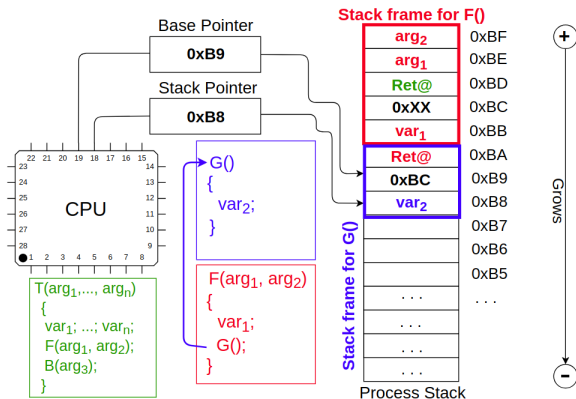
Next, the previous function stack frame pointer (i.e., BP) is saved (pushed) onto the stack and the BP is updated.

Buffer overflow on the stack



One local variable `var2`, used by function `G()`, is pushed onto the stack.

Buffer overflow on the stack



At this stage, the stack frame of function `G()` is complete. In this case, to access the local variable `var2`, the BP pointer is used as follow: `[BP-1]`.

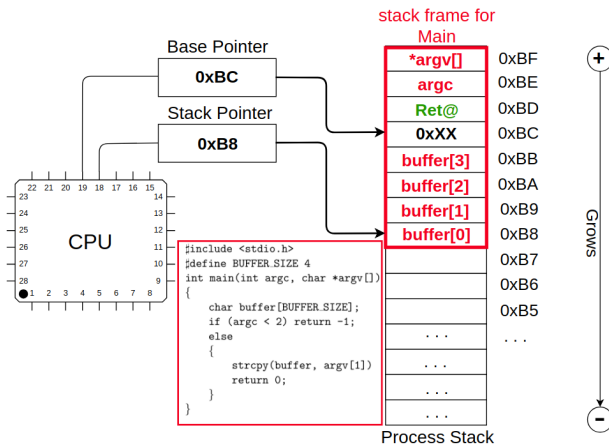
Note that if we were in a 32-bit computer, it will be `[BP-4]`.

Buffer overflow on the stack

Now, consider the following C program:

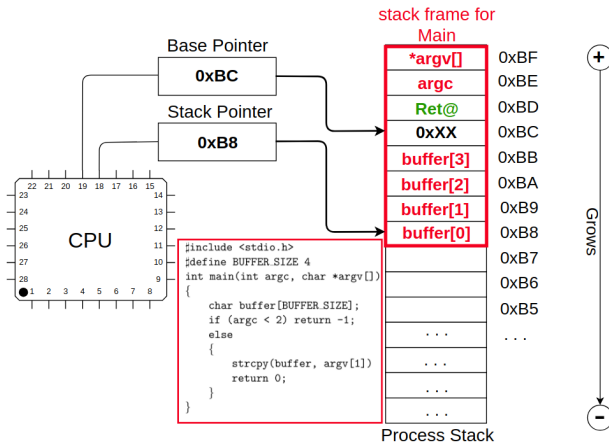
```
#include <stdio.h>
#define BUFFER_SIZE 4
int main(int argc, char *argv[])
{
    char buffer[BUFFER_SIZE];
    if (argc < 2) return -1;
    else
    {
        strcpy(buffer, argv[1]);
        return 0;
    }
}
```

Buffer overflow on the stack



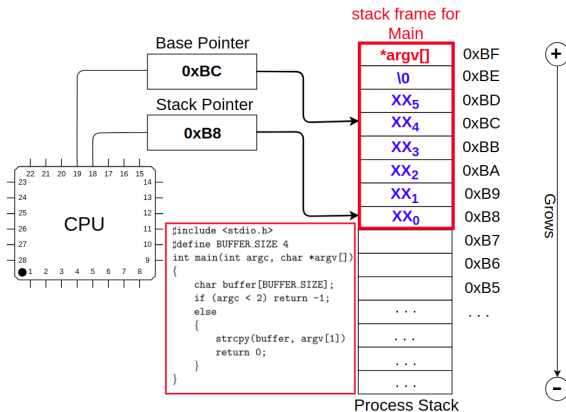
The stack will look like this.

Buffer overflow on the stack



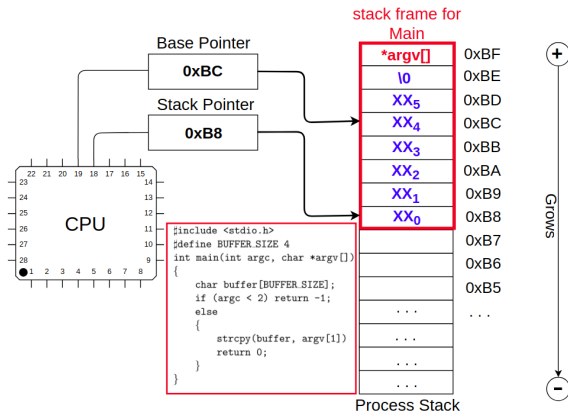
If the attacker inputs a string of 6 characters, such as XX₀...XX₅.

Buffer overflow on the stack



The stack will become:

Buffer overflow on the stack



Now, the goal of the attacker is then to overwrite the PC value with a value XX₅ that points to a malicious program, a.k.a., **Shellcode**.

Buffer overflow on the stack

Definition

Shellcode. Is a string that expresses a binary program that starts a shell interpreter. It is given as an input to a program to be copied into a buffer in such a way it overwrites what is beyond the buffer bound, in particular the return address from a function call.

The general term for any piece of code used to exploit a software vulnerability is **payload**.

Generally, for buffer overflow on the stack, it takes the following form:

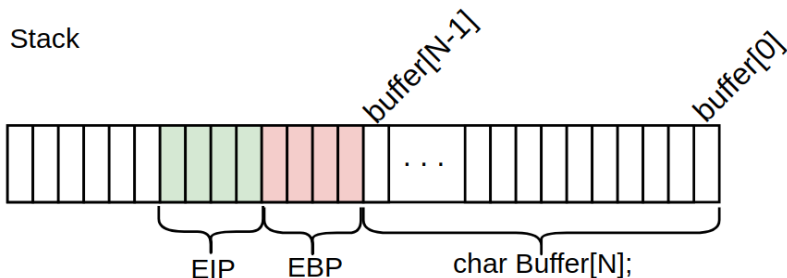
NOP : Shellcode : RET@ : \0

0x90 0x90 0x90 ... 0x90 : Shellcode : RET@ : \0

Where **RET@** refers to the initial address of the shellcode.

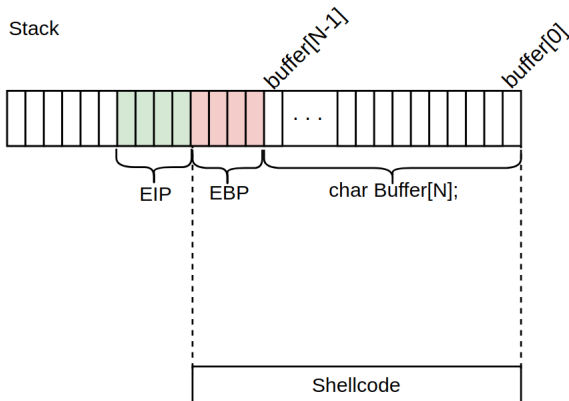
Buffer overflow on the stack

Consider a function stack frame which first local variable is a buffer of N chars. We also consider a 32-bit computer system and that the main memory is byte-addressable.



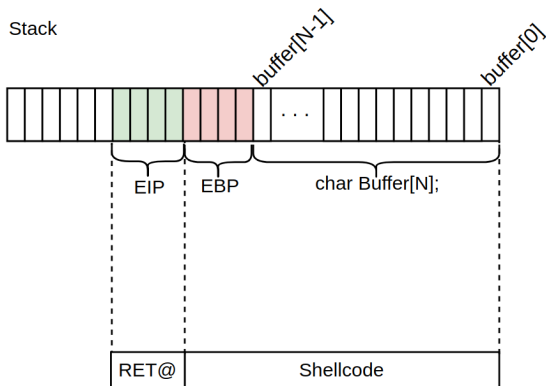
Buffer overflow on the stack

The attacker has to build a Shellcode that will be written into the buffer:



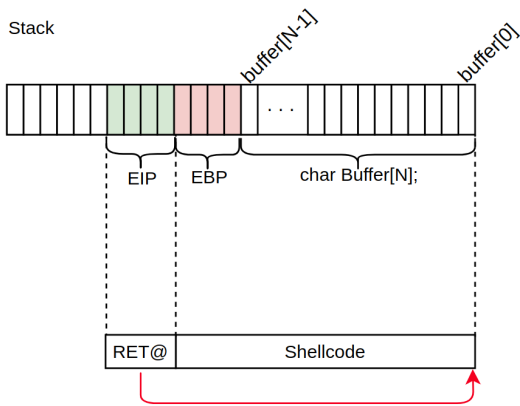
Buffer overflow on the stack

The string sequence that contains the Shellcode will also contain the address of the first character in the buffer as the new return address:



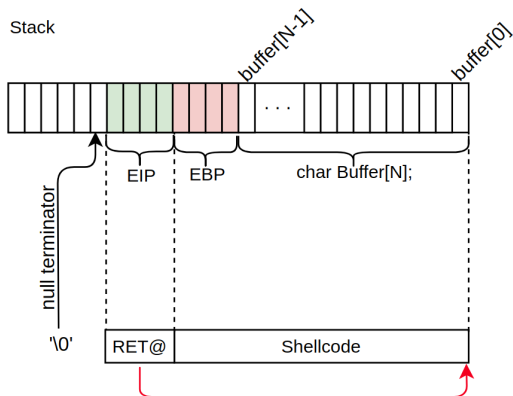
Buffer overflow on the stack

The new return address is actually the address of the first instruction of the Shellcode:



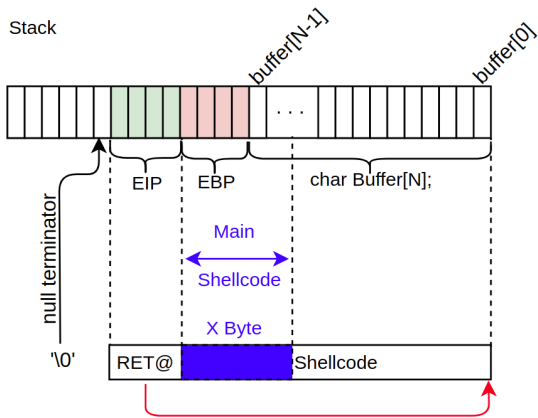
Buffer overflow on the stack

Since the attack has to provide the target program with a string sequence, the latter must end with a null terminator char:



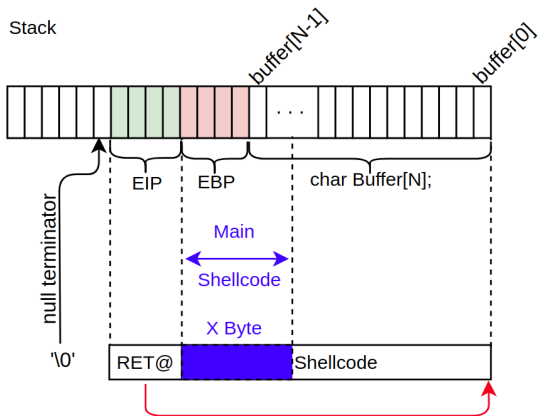
Buffer overflow on the stack

The Shellcode main function may be smaller than the total size of the buffer $X < N$:



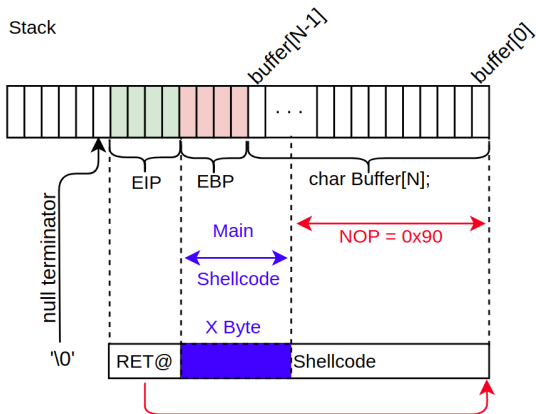
Buffer overflow on the stack

What will be the content of the buffer, from `buffer[0]` to `buffer[N-1+4-X]` if we want that content to be executable?

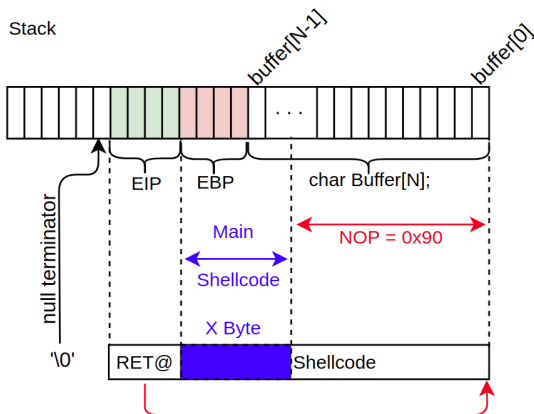


Buffer overflow on the stack

We can fill up that initial part with **NOP=0x90**.



Buffer overflow on the stack



What is the size of the payload if $N=256$ in this case, and how many NOP code (i.e., `0x90`) will be used if $X=200$?

Detect Buffer overflow

There are several mechanisms used to detect buffer overflow attack:

- Stack smashing prevention: Use canaries to detect PC overwriting attempts (use **-fno-stack-protector** to disable smash protection).

```
*** stack smashing detected ***: ./a.out terminated  
Aborted (core dumped)
```

- Disallow execution of code in a stack section of memory.
- Detect a sequence of NOP: 0x90 0x90 0x90 0x90 ... 0x90.
- Detect sequences such as /bin/bash or /bin/sh.

Overflow on the heap

Overflow on the heap. Consists of overwriting a command on the heap to execute an arbitrary malicious command.

Definition

The Heap. Is a memory segment allocated by the OS to each program in order to perform dynamic allocation during runtime.

Example.

```
Void Main ( )  
{  
    int c = 0;  
    int *p;  
    p = (int*) malloc(sizeof(int));  
    free(p);  
}
```

Overflow on the heap

Example of heap overflow (on Windows).

```
Void Main (int argc, char *argv[])
{
    char *t, *c;
    t = (char *) malloc(3);
    c = (char *) malloc(70);
    strcpy(c,"legitimate command")
    strcpy(t, argv[1]);
    system(c);
    return;
}
```

Integer Overflow

Integer overflow. Occurs when the variable type `int` is used instead of `unsigned_int`. It allows an attacker to use values that are not allowed but still getting access to some resources.

Observe the following program:

```

1 #include <stdio.h>
2
3 void main ()
4 {
5     int a;
6     int b = 100;
7     printf("Input value that is less than 10 if you want to win:\n");
8     scanf("%d", &a);
9     if(a*b<=1000)
10    {
11        printf("You won: %d dollars.\n",a*10);
12    }
13    else
14    {
15        printf("You lost %d dollars.\n",a);
16    }
17 }

```

Is it true that the maximum amount of money that can be won is: 100\$?

- End.